# getting-started

# Getting Started

Preston-Check runs a battery of pre-deployment security checks against your source code, on your machine, with no signup or telemetry by default. This guide walks you through installation, your first scan, CI integration, and the upgrade paths to Pro and Enterprise tiers.

## Install

Pick the channel that fits your environment.

**Homebrew** (macOS or Linux with brew):

```
brew tap preston-check/preston-check
brew install preston-check
```

**Docker** (any system with Docker):

```
docker run --rm -v "$(pwd):/src" ghcr.io/preston-check/scan:latest
```

Mount your project at `/src` and the scanner runs against it. Add `-v ~/.preston-check:/home/preston/.preston-check` to share license files into the container.

**Curl-bash installer** (works anywhere with curl):

```
curl -fsSL https://get.preston-check.com/install.sh | sh
```

The installer fetches the latest release tarball, verifies its SHA-256 against the `.sha256` sidecar published alongside, and installs the catalog under `$PREFIX/share/preston-check` with a thin shim at `$PREFIX/bin/preston-check`. Defaults to `/usr/local`; override with `PRESTON_PREFIX=$HOME/.local`. Pin a specific version with `PRESTON_VERSION=v1.7.3`.

**GitHub Action** (in your CI):

Add to `.github/workflows/preston-check.yml`:

```yaml
- uses: preston-check/scan-action@v1
  with:
    fail-on: high
    report-path: preston-check-report.md
```

See `.github/workflows/preston-check.yml` in the repository for a complete example.

# Run your first scan

In your project directory:

```
preston-check
```

You will see a banner identifying the detected language, license tier (Free if you have no license), and OSS exemption (Pro features are granted free for repositories with a recognized OSS LICENSE). Each check prints a colored line — green PASS, red FAIL, yellow WARN, blue SKIP — and a final summary shows totals.

To save a report:

```
preston-check --report security-audit.md
```

The markdown report is suitable for code review attachments and PR comments. If `pandoc` and Chrome are available, Preston-Check also generates a PDF version next to the markdown file.

To run a faster check focused on the original 20 categories (approximately 30 seconds):

```
preston-check --light
```

To run a single check by ID:

```
preston-check --check 07-blacklist-check
```

To list all available checks:

```
preston-check --list
```

# AI-augmented findings

When `ANTHROPIC_API_KEY` or `OPENAI_API_KEY` is set, the runner can attach a per-finding AI assessment (false-positive classification, plain-English explanation, suggested fix) to the report addendum.

```
# Analysis only — adds explanations and false-positive flags
preston-check --ai-augment

# Analysis plus suggested unified-diff patches per finding
preston-check --ai-fix
```

Both flags are no-ops under `--airgap` and graceful no-ops without a configured API key. Per-finding responses are cached under `~/.preston-check/ai-cache/` so reruns over unchanged code are free. Local Ollama is also supported via `PRESTON_AI_PROVIDER=ollama`.

# Examples

Drop-in usage patterns for the four most-requested CI surfaces live at `examples/`:

- `examples/github-action.yml` — PR security gate
- `examples/gitlab-ci.yml` — GitLab CI integration
- `examples/circleci-config.yml` — CircleCI integration
- `examples/pre-commit-hook.sh` — local pre-commit gate

# Project configuration

For repeatable runs, create a config file (e.g., `.preston-check.yml` at your repo root):

```
app_name: my-payment-service
source_dir: .
log_dir: /var/log/myapp
ssh_host: prod-server-01
api_base_url: https://api.example.com
redis_host: localhost
db_host: db.example.com
```

Then run:

```
preston-check --config .preston-check.yml
```

The only required field is `source_dir`. Live-monitoring checks (P-10) require `ssh_host`. Other fields are used by specific check categories and degrade to SKIP when missing.

# CI integration

For GitHub Actions, drop the example workflow from this repository into your project. The Action posts a security-score summary as a PR comment by default and uploads the full markdown report as a build artifact.

For GitLab CI:

```
preston_check:
  image: ghcr.io/preston-check/scan:latest
  script:
    - preston-check --ci --report preston-check-report.md
  artifacts:
    when: always
    paths: [preston-check-report.md]
```

For pre-push git hooks:

```
echo 'preston-check --light --ci' >> .git/hooks/pre-push
chmod +x .git/hooks/pre-push
```

The `--ci` flag exits with code 1 on any FAIL, blocking the push. Use `--ci-soft` if you want the scanner to never exit non-zero (useful when the calling system applies its own threshold logic).

# Privacy and airgap mode

Preston-Check reads source files locally and makes no network calls unless you explicitly opt in to anonymous telemetry. For environments that require absolute network isolation (e.g., regulated production infrastructure), pass `--airgap`:

```
preston-check --airgap --report report.md
```

This disables every potential outbound code path including the opt-in telemetry. The scanner is open-source so you can verify these guarantees in `lib/telemetry.sh` directly — every network call goes through that file, and `--airgap` short-circuits it.

# Filter by framework, severity, or category

The catalog of 236 checks can be scoped at runtime. The four filter flags compose freely:

```
# Audit prep for a specific regulator
preston-check --framework MiCA --report mica.md
preston-check --framework DORA --report dora.md
preston-check --framework NYDFS --report nydfs.md
preston-check --framework "PCI-DSS" --report pci.md

# Filter by check type
preston-check --code-only       # only static source-code analysis
preston-check --docs-only       # only documentation / evidence verification
preston-check --infra-only      # only infrastructure / config scans
preston-check --live-only       # only SSH-based production log checks

# Filter by severity
preston-check --critical-only   # ~8 checks, ~12 second runtime
preston-check --high-and-up     # critical + high (~73 checks, CI gating)
preston-check --severity medium # only medium-severity checks

# Combinations
preston-check --framework DORA --docs-only         # DORA evidence prep only
preston-check --framework "OWASP-LLM-Top-10" --code-only
preston-check --critical-only --ci                 # fast CI gate
```

See `docs/all-frameworks.md` for the full list of supported framework filters with per-check coverage.

# Free tier vs. Pro vs. Enterprise

The Free tier requires no license, no email, and no signup. It covers all 100+ scanning categories — every P-XX check that examines source code. This is the same scanner that paying customers use; the difference is in what surrounds it.

Pro tier ($999/repo/yr or $4,999/yr unlimited) adds the compliance- evidence layer (P-83 through P-95 evidence verification, plus the auditor-ready packaging that bundles findings into a deliverable), multi-repo dashboard for tracking score across an organization, and branded reports with priority email

support. Most fintechs preparing for SOC 2 or PCI-DSS audits land here.

Enterprise tier ($29,999+/yr starting) adds white-label reports (your branding instead of Preston-Check), SSO, custom check authoring with maintainer support, signed audit packages, and a dedicated customer-success contact.

If your repository has a recognized OSS LICENSE file (MIT, Apache, BSD, GPL, MPL, ISC, Unlicense), the scanner automatically grants you Pro features for free. Public security work makes the community stronger.

To install a Pro/Enterprise license:

```
mkdir -p ~/.preston-check
cp /path/to/your.license ~/.preston-check/license
preston-check  # license is auto-detected
```

Or set `PRESTON_LICENSE=/path/to/your.license` in your environment.

# Contributing

If you find a security pattern Preston-Check doesn't catch, the community contribution path is documented in `CONTRIBUTING.md`. The short version: copy `templates/check.sh` to `checks/community/proposed/<NUMBER>-<short-name>.sh`, fill in the metadata block, write your detection logic, run `tools/lint-check.sh path/to/your-check.sh`, and open a pull request. Maintainer review promotes the check from `proposed` to `accepted`, and field validation eventually promotes it to `verified`. Author attribution appears in every report that fires your check.